HDOC - Day 16 Activity

C Programming Assignment

Name-Arushi Gusain

Sap id-590033489

Batch 15

Date- 9/09/2026

Topic: Menu-driven digit operations using switch-case and separate void functions

Problem Requirements

Write a menu-driven C program for a positive integer. Use switch-case. Each menu option must call a separate
user-defined function having void return type and no arguments. The selected function itself reads the number,
performs the operation, and displays the result.
1. Check whether digits are in increasing order from left to right.
2. Check whether digits are in decreasing order from left to right.
3. Find the largest and smallest digit.
4. Find the second largest and second smallest distinct digits.

Algorithm

Main Algorithm

1. Start.
2. Display four menu options.
3. Read the user's choice.
4. Use switch(choice):

1 -> call checkIncreasing()
2 -> call checkDecreasing()

3 -> call findLargestSmallest()
4 -> call findSecondLargestSmallest()
default -> display "Invalid choice".
5. Stop.

Function checkIncreasing()

1. Read a positive integer n.
2. If n <= 0, display an error and return.
3. Compare adjacent digits from right to left. For increasing left-to-right order,
each digit on the left must be smaller than the digit on its right.
4. If every comparison satisfies this condition, print Increasing Order; otherwise print Not in Increasing Order.

Function checkDecreasing()

1. Read a positive integer n and validate it.
2. Compare adjacent digits. For decreasing left-to-right order, each digit on the left must be greater than the digit on its right.
3. Display the result.

Function findLargestSmallest()

1. Read a positive integer n and validate it.
2. Initialize smallest = 9 and largest = 0.
3. Extract each digit using n % 10; update smallest/largest; remove the digit using n /= 10.
4. Display smallest and largest.

Function findSecondLargestSmallest()

1. Read a positive integer n and validate it.
2. Mark the digits present using an array present[10].
3. Scan 0..9 to obtain the smallest and second smallest distinct digits.
4. Scan 9..0 to obtain the largest and second largest distinct digits.
5. If fewer than two distinct digits exist, display that a second value does not exist; otherwise display both second values.

Test Case Table

Case	Choice	Input	Expected Output	Result
1	1	123459	Digits are in increasing order	Pass
2	1	1229	Digits are NOT in increasing order	Pass
3	2	975310	Digits are in decreasing order	Pass
4	2	9771	Digits are NOT in decreasing order	Pass
5	3	583204	Largest = 8; Smallest = 0	Pass
6	4	583204	Second largest = 5; Second smallest = 2	Pass
7	4	7777	Second largest/smallest do not exist	Pass
8	5	-	Invalid choice	Pass

C Program

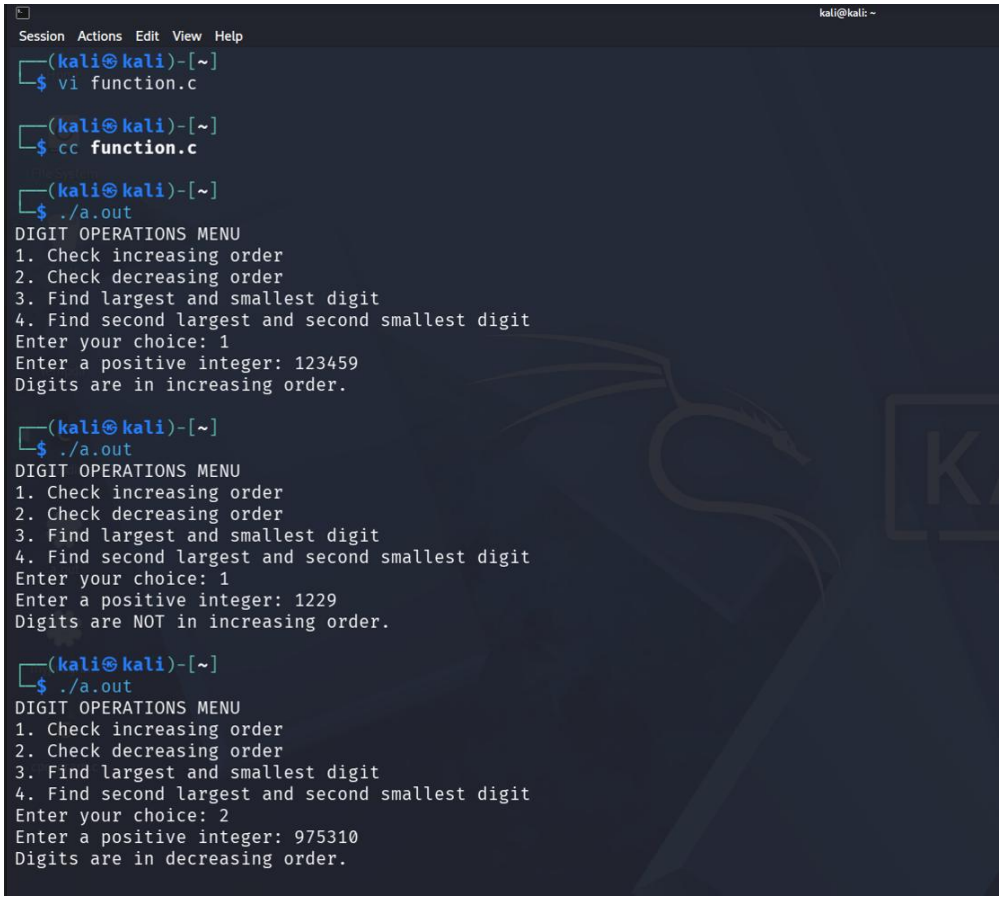
```
#include <stdio.h>
void checkIncreasing(void);
void checkDecreasing(void);
void findLargestSmallest(void);
void findSecondLargestSmallest(void);
int main(void)
{
int choice;
printf("DIGIT OPERATIONS MENU\n");
printf("1. Check increasing order\n");
printf("2. Check decreasing order\n");
printf("3. Find largest and smallest digit\n");
printf("4. Find second largest and second smallest digit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1: checkIncreasing(); break;
case 2: checkDecreasing(); break;
case 3: findLargestSmallest(); break;
case 4: findSecondLargestSmallest(); break;
default: printf("Invalid choice.\n");
```

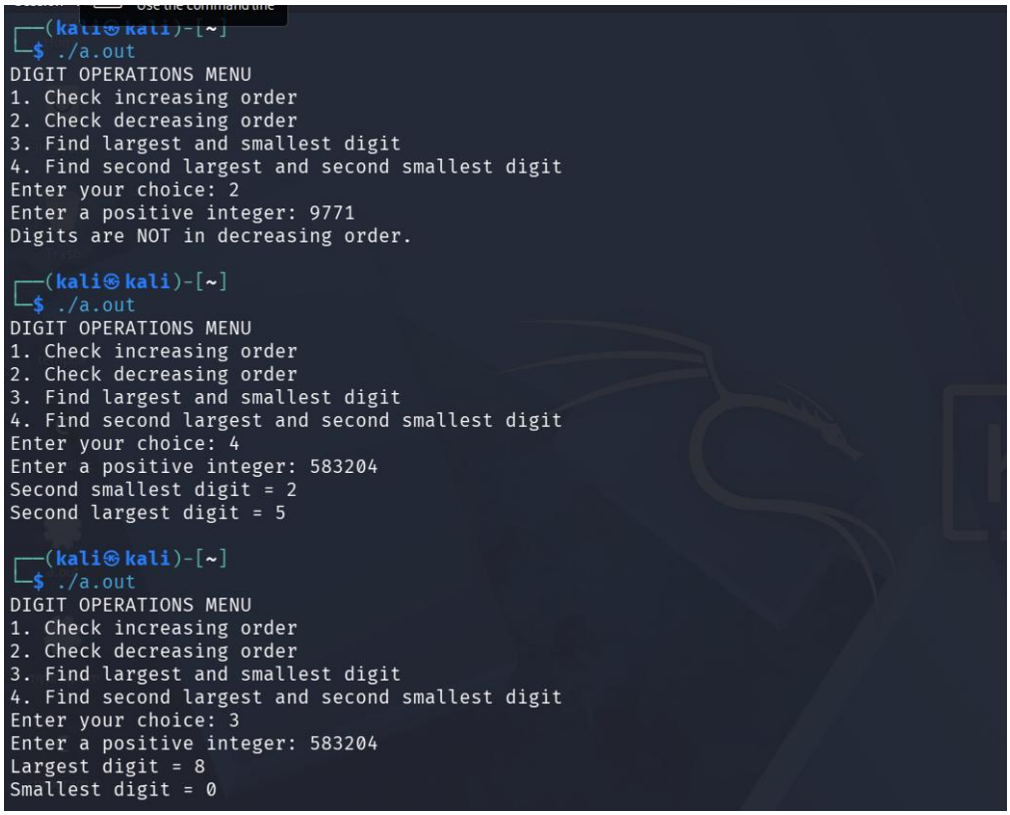
```
}
return 0;
}
void checkIncreasing(void)
{
long long n, temp;
int right, left, increasing = 1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) { printf("Invalid input. Enter a positive integer.\n"); return; }
temp = n;
right = temp % 10;
temp /= 10;
while (temp > 0) {
left = temp % 10;
if (left >= right) { increasing = 0; break; }
right = left;
temp /= 10;
}
if (increasing) printf("Digits are in increasing order.\n");
else printf("Digits are NOT in increasing order.\n");
}
void checkDecreasing(void)
{
long long n, temp;
int right, left, decreasing = 1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) { printf("Invalid input. Enter a positive integer.\n"); return; }
temp = n;
right = temp % 10;
temp /= 10;
while (temp > 0) {
left = temp % 10;
if (left <= right) { decreasing = 0; break; }
```

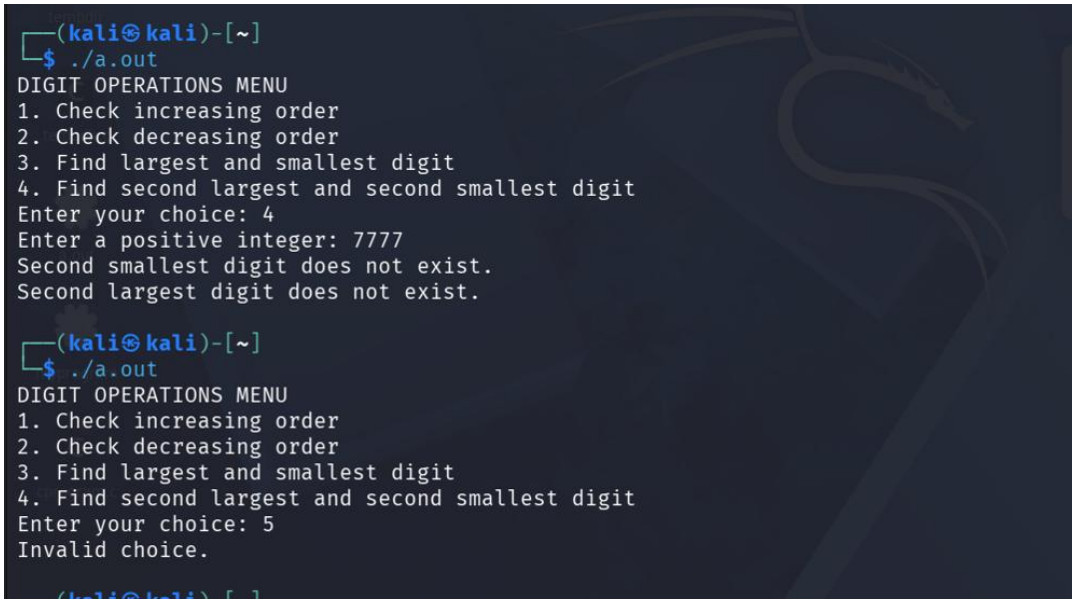
```
right = left;
temp /= 10;
}
if (decreasing) printf("Digits are in decreasing order.\n");
else printf("Digits are NOT in decreasing order.\n");
}
void findLargestSmallest(void)
{
long long n, temp;
int digit, largest = 0, smallest = 9;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) { printf("Invalid input. Enter a positive integer.\n"); return;
}
temp = n;
while (temp > 0) {
digit = temp % 10;
if (digit > largest) largest = digit;
if (digit < smallest) smallest = digit;
temp /= 10;
}
printf("Largest digit = %d\n", largest);
printf("Smallest digit = %d\n", smallest);
}
void findSecondLargestSmallest(void)
{
long long n, temp;
int present[10] = {0}, digit, i;
int smallest = -1, secondSmallest = -1;
int largest = -1, secondLargest = -1;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) { printf("Invalid input. Enter a positive integer.\n"); return; }
temp = n;
while (temp > 0) {
```

```
digit = temp % 10;
present[digit] = 1;
temp /= 10;
}
for (i = 0; i <= 9; i++) {
if (present[i]) {
if (smallest == -1) smallest = i;
else { secondSmallest = i; break; }
}
}
for (i = 9; i >= 0; i--) {
if (present[i]) {
if (largest == -1) largest = i;
else { secondLargest = i; break; }
}
}
if (secondSmallest == -1)
printf("Second smallest digit does not exist.\n");
else
printf("Second smallest digit = %d\n", secondSmallest);
if (secondLargest == -1)
printf("Second largest digit does not exist.\n");
else
printf("Second largest digit = %d\n", secondLargest);
}
```

Compilation , Execution & Output







Conclusion

The program successfully uses switch-case and four separate void user-defined functions to perform the required digit-based operations. It validates positive input, checks strict increasing/decreasing digit order, finds extreme digits, and finds the second largest and second smallest distinct digits.